

PRODUCT REQUIREMENTS DOCUMENT

SafeSite AI

Real-Time Construction Site Safety Monitoring & Compliance System

Version 1.0 | April 2026

AI-Powered Solution Expo — SRM Institute of Science and Technology
Delhi-NCR Campus, Ghaziabad

Domain	Computer Vision, Deep Learning, Industry 4.0
Project Status	In Progress
Demo Date	16 April 2026, 2:00 PM — 4:00 PM
Venue	Ada Lovelace / John McCarthy Seminar Hall

Table of Contents

- 1. Overview
- 2. Problem Statement
- 3. Proposed Solution
- 4. Core Features
- 5. Feature Details
- 6. System Architecture
- 7. Technology Stack
- 8. User Roles & Personas
- 9. User Flows
- 10. Non-Functional Requirements
- 11. Success Metrics
- 12. Risks & Mitigations
- 13. Development Phases
- 14. Future Scope

1. Overview

SafeSite AI is a real-time construction site safety monitoring system that uses computer vision and deep learning to detect safety violations as they happen. It processes live video feeds, identifies workers who aren't wearing the required protective equipment, flags unauthorized entry into restricted zones, generates incident reports automatically, announces violations through a voice alert system, and presents all of this through an analytics dashboard that helps site managers make data-driven safety decisions. The goal is straightforward — replace the manual, unreliable, after-the-fact approach to construction safety with something that's intelligent, automated, and always watching.

2. Problem Statement

Construction sites are among the most dangerous workplaces in India. Thousands of workers die or get seriously injured every year, and a large chunk of these incidents happen simply because someone wasn't wearing a helmet, a vest, or a harness. The rules exist on paper, but enforcement is a different story — it depends entirely on human supervisors who can't possibly keep their eyes on every worker across a sprawling, chaotic site all day long. Violations slip through, nobody documents them, and nothing changes until someone actually gets hurt. There's also no proper system to track which parts of a site are the riskiest, flag workers who repeatedly ignore safety norms, or give managers clear, data-backed reports. The whole process is manual, patchy, and only kicks in after the damage is done.

3. Proposed Solution

SafeSite AI is built to fix this. It's a web-based system that takes in a live video feed from a construction site camera and runs it through a YOLOv8 deep learning model to detect whether workers are wearing the required safety gear — helmets, vests, harnesses. The moment someone is spotted without proper PPE, the system draws a bounding box around them on the video, flags the violation, and logs it with a timestamp and an auto-captured screenshot.

But it doesn't stop at just spotting violations. Site managers can draw restricted or high-risk zones directly on the camera feed, and the system triggers an alert the moment someone walks into those areas without authorization. Every violation that gets detected is logged automatically, and the system compiles these into ready-to-share PDF incident reports — no manual paperwork needed. It also has a voice alert feature that announces violations out loud in Hindi and English through text-to-speech, almost like having a PA system that reacts instantly. On top of all this, there's an analytics dashboard that shows violation trends over time, highlights the most dangerous zones, identifies when violations peak during the day, and gives managers the kind of data they need to actually make informed decisions about safety.

The whole idea is to take construction site safety from being a reactive, human-dependent mess and turn it into something intelligent, automated, and driven by real data — which is exactly what Industry 4.0 is about.

4. Core Features

#	Feature	Description	Priority
---	---------	-------------	----------

1	Real-Time PPE Detection	Detect missing helmets, vests, and harnesses from live video using YOLOv8	P0 — Must Have
2	Zone Intrusion Detection	Allow managers to define restricted areas on the feed; alert on unauthorized entry	P0 — Must Have
3	Automated Incident Reports	Auto-generate PDF reports with timestamps, screenshots, and violation details	P0 — Must Have
4	Voice Alert System	Announce violations in Hindi and English via text-to-speech in real time	P1 — Should Have
5	Analytics Dashboard	Visualize violation trends, peak hours, high-risk zones, and compliance metrics	P1 — Should Have

5. Feature Details

5.1 Real-Time PPE Detection

This is the backbone of the entire system. A live video feed — from a webcam, IP camera, or even a pre-recorded video for demo purposes — is processed frame by frame through a YOLOv8 object detection model trained to recognize PPE items like helmets and safety vests. When a worker is detected without the required gear, the system draws a bounding box around them on the live feed, labels the violation type, and immediately triggers the logging pipeline. Detection happens in real time, meaning there's no delay between a violation occurring and the system catching it. The model runs inference using OpenCV for frame capture and the Ultralytics YOLOv8 library for detection.

5.2 Zone Intrusion Detection

Construction sites have areas that are off-limits — near heavy machinery, open excavations, electrical panels, unstable structures. In SafeSite AI, a site manager can open the web interface and draw polygons directly on the camera feed to mark these restricted zones. Once a zone is defined, the system monitors whether any detected person enters that zone. If someone crosses into a restricted area, the system flags it as a zone intrusion, captures a screenshot, and fires an alert. This works independently of PPE detection — even a fully geared worker entering a restricted zone without clearance gets flagged. The zone definitions are saved and persist across sessions.

5.3 Automated Incident Report Generation

Every violation that the system catches — whether it's a missing helmet or a zone intrusion — gets logged into a local database with a timestamp, violation type, screenshot, and zone information (if applicable). At any point, the site manager can generate a PDF incident report covering a specific time period. The report is built using ReportLab and includes a summary of all violations, individual violation entries with their screenshots, and basic statistics like total violations by type and by zone. The report is formatted professionally and ready to hand over to compliance officers, site owners, or safety auditors. No one has to sit down and write anything manually.

5.4 Bilingual Voice Alert System

When a violation is detected, the system doesn't just log it silently. It announces it out loud using text-to-speech via the pyttsx3 library. The alert is spoken in both Hindi and English, making it accessible to the actual workforce on the ground — most of whom may not be monitoring a screen. Think of it as an automated PA system. For example, if a worker is detected without a helmet in Zone B, the system might announce: "Warning — helmet violation detected in Zone B" followed by the Hindi equivalent. The voice alerts run on a separate thread so they don't block the video processing pipeline.

5.5 Analytics Dashboard

All the violation data the system collects feeds into an analytics dashboard built with Plotly and rendered through the Flask web interface. The dashboard shows violation trends over time (hourly, daily, weekly), a breakdown of violations by type (helmet, vest, zone intrusion), a heatmap or bar chart showing which zones have the most incidents, peak violation hours during the day, and overall compliance rates. This gives site managers something they've never had before — actual data to base their safety decisions on, rather than gut feeling or sporadic supervisor reports.

6. System Architecture

The system follows a modular pipeline architecture with five main components that work together but are loosely coupled enough to be developed and tested independently.

Layer	Component	Technology	Responsibility
Input	Video Capture Module	OpenCV	Capture frames from webcam, IP camera, or video file
Processing	Detection Engine	YOLOv8, Ultralytics	Run inference on each frame, detect PPE and persons
Processing	Zone Monitor	CVZone, NumPy	Check if detected persons fall within restricted polygons
Alert	Voice Engine	pyttsx3	Announce violations in Hindi and English via TTS
Storage	Violation Logger	SQLite / JSON	Store violation records with timestamps and screenshots
Output	Report Generator	ReportLab	Compile violation data into downloadable PDF reports
Output	Web Dashboard	Flask, Plotly	Serve live feed, zone config, analytics, and report downloads

7. Technology Stack

Category	Technology	Purpose
Language	Python 3.10+	Primary development language
Object Detection	YOLOv8 (Ultralytics)	Real-time PPE and person detection
Computer Vision	OpenCV, CVZone	Frame capture, drawing, zone logic
Web Framework	Flask	Serve web UI, API endpoints, video stream
Frontend	HTML, CSS, JavaScript	Dashboard UI, zone drawing interface
Text-to-Speech	pyttsx3	Bilingual voice alerts (Hindi / English)
PDF Generation	ReportLab	Automated incident report creation
Data Visualization	Matplotlib, Plotly	Charts and graphs for analytics dashboard
Data Handling	NumPy, SQLite	Array operations, local violation storage

8. User Roles & Personas

Site Manager / Safety Officer

This is the primary user. They configure the system — setting up camera feeds, drawing restricted zones, and defining which PPE items to monitor. They receive real-time alerts, review violations on the dashboard, and generate incident reports for compliance audits. They care about reducing accidents, meeting regulatory requirements, and having data to justify safety investments to higher management.

Site Supervisor / Foreman

A secondary user who monitors the live feed during their shift. They respond to voice alerts by taking immediate corrective action on the ground — telling a worker to put on a helmet, clearing people from a restricted zone. They don't configure the system but rely on it to catch what they might miss.

Construction Workers

They don't interact with the system directly, but they're the ones being monitored and, more importantly, the ones being protected. The voice alerts serve as direct, immediate feedback to them in a language they understand.

9. User Flows

Flow 1: Setting Up Monitoring

The site manager opens the web interface, selects a camera source (webcam or IP camera URL), and starts the live feed. They then switch to the zone configuration tab, draw polygons over restricted areas on the feed, label each zone, and save. The system is now actively monitoring both PPE compliance and zone intrusions.

Flow 2: Violation Detection and Alert

A worker walks into the camera frame without a helmet. Within milliseconds, the YOLOv8 model detects the violation, draws a red bounding box on the feed, and the violation logger captures a screenshot with a timestamp. Simultaneously, the voice engine announces the violation in Hindi and English. The violation appears as a new entry in the live violation feed on the dashboard.

Flow 3: Generating a Report

At the end of a shift or day, the site manager navigates to the reports section, selects a date range, and clicks generate. The system compiles all violations in that period into a PDF with violation summaries, individual entries with screenshots, and statistics. The manager downloads the report and shares it with the compliance team or site owner.

Flow 4: Reviewing Analytics

The manager opens the analytics dashboard to review the past week. They see that Zone C near the excavation site had the highest number of intrusions, helmet violations peak between 2 PM and 4 PM (likely due to heat), and overall compliance has improved 12% since the system was deployed. They use this data to schedule additional safety briefings during peak violation hours.

10. Non-Functional Requirements

Requirement	Target	Notes
Detection Latency	Under 500ms per frame	Real-time feel; depends on GPU availability
Frame Rate	15-30 FPS	Smooth enough for live monitoring; adjustable
Detection Accuracy	mAP > 0.75	Using pre-trained + fine-tuned YOLOv8 model
Voice Alert Delay	Under 2 seconds	From detection to audible announcement
Report Generation	Under 10 seconds	For a report covering up to 500 violations
Browser Support	Chrome, Firefox, Edge	Modern browsers with JavaScript enabled
Concurrent Users	1-3 dashboard users	Prototype scope; not designed for scale

11. Success Metrics

For the demo at the expo, success means the system can reliably detect PPE violations from a live webcam feed in real time, correctly identify zone intrusions when someone enters a marked area, generate a clean PDF report on demand, announce violations audibly in both Hindi and English, and display meaningful analytics on the dashboard. The detection should be visibly accurate to the judges — bounding boxes should appear on the right people, false positives should be minimal, and the end-to-end flow from detection to alert to report should feel seamless. Beyond the demo, the real success metric is whether a site manager watching the demo thinks: "I'd actually use this on my site."

12. Risks & Mitigations

Risk	Impact	Mitigation
No GPU on demo laptop	Slow inference, laggy feed	Use YOLOv8n (nano) variant; pre-test on CPU; keep backup video
Poor lighting at venue	Low detection accuracy	Test with varied lighting; use pre-recorded video as fallback
False positives on stage	Looks unreliable to judges	Fine-tune confidence threshold; rehearse demo scenario
Audio issues for voice alerts	Judges can't hear TTS	Bring external speaker; test audio setup before demo
Webcam not detected	No live feed	Have pre-recorded construction video ready as input

13. Development Phases

Phase	Timeline	Deliverables
Phase 1: Core Detection	Day 1-2	YOLOv8 model integration, OpenCV video pipeline, basic helmet/vest detection with bounding boxes on live feed
Phase 2: Zone System	Day 2-3	Zone drawing UI on camera feed, polygon-based intrusion detection logic, zone persistence
Phase 3: Logging & Reports	Day 3	Violation logging with timestamps and screenshots, PDF report generation via ReportLab
Phase 4: Voice Alerts	Day 3-4	pyttsx3 integration, bilingual alert messages, threaded audio to avoid blocking
Phase 5: Dashboard	Day 4	Flask web UI, Plotly charts for trends/zones/peak hours, live violation feed
Phase 6: Polish & Demo Prep	Day 5	UI polish, demo rehearsal, edge case handling, backup video preparation

14. Future Scope

The current prototype is scoped for a single-camera, single-site demo. But the architecture is designed so it can grow. Multi-camera support is the most natural next step — feeding multiple streams into the same dashboard so a manager can monitor an entire site from one screen. Worker identification using face recognition or badge detection would enable tracking repeat offenders and tying violations to specific individuals. Integrating with IoT sensors — like wearable devices that detect falls or proximity to heavy equipment — would add another layer of safety beyond what cameras alone can see. Pushing alerts to mobile phones via SMS or WhatsApp would ensure managers are notified even when they're not looking at the dashboard. And with enough violation data collected over time, predictive analytics could start forecasting when and where the next violation is likely to happen, shifting the system from reactive monitoring to genuinely preventive safety management.